Foundations of Machine Learning

Assignment 2

Name: Hemachandra

Roll Number: MA25C019

Course Code: DA5400

Question 1

You are given a data-set with 400 data points in $\{0,1\}^{50}$ generated from a mixture of some distribution in the file `A2Q1.csv`. **Hint:** Each data point is a flattened version of a $\{0,1\}^{10\times5}$ matrix.

(i)

Determine which probabilistic mixture could have generated this data (it is not a Gaussian mixture). Derive the EM algorithm for your choice and show your calculations. Write a piece of code to implement the algorithm you derived by setting the number of mixtures $K = 4$. Plot the log-likelihood (averaged over 100 random initializations) as a function of iterations.

Ans: Each point in the dataset is a 50×1 vector, each entry either 0 or 1. We can assume that each entry was generated from a Bernoulli distribution with mean p and variance $p(1-p)$ and for simplicity we can also assume each entry was generated independent of the other entries. Now we need to determine if each entry in the vector is identically distributed or not.
We can do this by assuming that each entry in the vector is identically distributed and there are 50 samples, k^{th} entry of the each vector belong to the kth sample. By the central limit theorem the sample proportion follows a normal distribution hence if these sample proportions are not normally distributed we are safe to say the entries are not identically distributed.
For this the file '1a.py' has a piece of code which computes the vector P where each kth entry is the sample proportion of the kth sample,

$$P = \frac{1}{m}\sum_{i=1}^{m} x_i$$

where m is the number of datapoints, x_i is a 50×1 vector. where the vector.
We get the mean of the sample proportion as 0.3371 and the variance as 0.111. But the computed sample proportions are more dense at the extremes than at the mean. Hence we are safe to conclude the entries are not identically distributed.

```
Assume each entry is identically distributed bernoulli random variable
Sample Proportion for each entry
[0.0875 0.0925 0.1075 0.085  0.095  0.115  0.1125 0.0825 0.09   0.1225
 0.1125 0.13   0.7125 0.8725 0.9    0.9175 0.885  0.8875 0.915  0.08
 0.0975 0.12   0.685  0.275  0.86   0.2975 0.7    0.0775 0.1025 0.105
 0.0825 0.1275 0.685  0.89   0.8875 0.2975 0.6775 0.7225 0.7    0.12
 0.0825 0.1    0.075  0.115  0.09   0.12   0.1    0.085  0.0825 0.0925]
Average of distribution of sample proportion
0.33709999999999996
Variance of distribution of sample proportion
0.11089550000000004
```

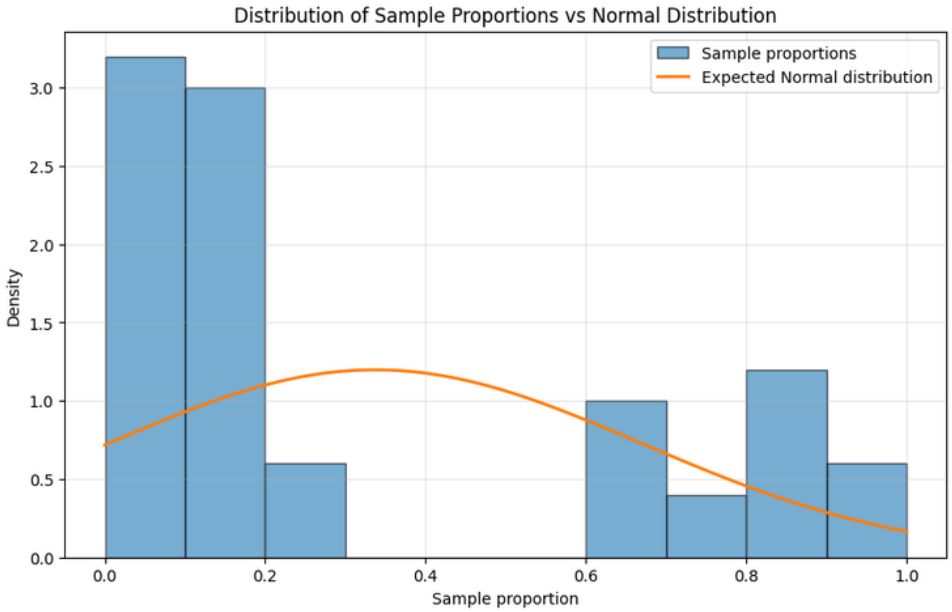


Figure 1: Sample Proportions of each entry

Thus for each kth entry in the vector we need its own parameter p_k. And since we have already assumed independence of the entries the mass function looks like

$$f(x_{(1,1)}, ..., x_{m,n}) = \prod_{k=1}^{n} p_k^{x_{ik}} (1 - p_k)^{(1-x_{ik})}$$

where n is the length of the vector and x_{ik} is the kth entry of the ith data point.
Since we want to model a mixture model with K mixtures, we can write the mass function as

$$f(x_{(1,1)}, ..., x_{(m,n)}) = \sum_{j=1}^{K} \theta_j \prod_{k=1}^{n} p_{jk}^{x_{ik}} (1 - p_{jk})^{(1-x_{ik})}$$

where p_{jk} is the parameter for the kth entry of the vector in the jth mixture.
Now we want to derive the EM Algorithm to get the parameters that maximizes the likelihood of observing the data.
The likelihood function looks like

$$L(\theta_1, ..., \theta_K, p_{(1,1)}, ..., p_{(K,n)}) = \prod_{i=1}^{m} \sum_{j=1}^{K} \theta_j \prod_{k=1}^{n} p_{jk}^{x_{ik}} (1 - p_{jk})^{(1-x_{ik})}$$

The Log likelihood looks like

$$l(\theta,p) = logL(\theta,p) = \sum_{i=1}^{m} log(\sum_{j=1}^{K} \theta_j \prod_{k=1}^{n} p_{jk}^{x_{ik}} (1-p_{jk})^{(1-x_{ik})})$$

For each x_i introduce $\lambda_{i1}, ..., \lambda_{iK}$ with $\sum_{j=1}^{K} \lambda_{ij} = 1$ and $0 \leq \lambda_{ij} \leq 1$

$$l(\theta,p) = \sum_{i=1}^{m} log(\sum_{j=1}^{K} \lambda_{ij} \frac{\theta_j}{\lambda_{ij}} \prod_{k=1}^{n} p_{jk}^{x_{ik}} (1-p_{jk})^{(1-x_{ik})})$$

log is a concave function, using Jensen's inequality

$$l(\theta,p) \geq \sum_{i=i}^{m} \sum_{j=1}^{K} \lambda_{ij} log(\frac{\theta_j}{\lambda_{ij}} \prod_{k=1}^{n} p_{jk}^{x_{ik}} (1-p_{jk})^{(1-x_{ik})}) = h(\theta,p)$$

Now simplifying $h(\theta,p)$

$$h(\theta,p) = \sum_{i=1}^{m} \sum_{j=1}^{K} \lambda_{ij} log\theta_j + \sum_{i=1}^{m} \sum_{j=1}^{K} \lambda_{ij} (\sum_{k=1}^{n} x_{ik} log p_{jk} + (1-x_{ik}) log(1-p_{jk})) - \sum_{i=1}^{m} \sum_{j=1}^{K} \lambda_{ij} log\lambda_{ij}$$

Fix λ_{ij}'s an maximize over p_{jk} and θ_j
To find p_{jk}

$$\frac{\delta h}{\delta p_{jk}} = \sum_{i=1}^{m} \lambda_{ij} (\frac{x_{ik}}{p_{jk}} - \frac{(1-x_{ik})}{(1-p_{jk})}) = 0$$

$$\implies \sum_{i=1}^{m} \lambda_{ij} x_{ik} (1-p_{jk}) = \sum_{i=1}^{m} \lambda_{ij} (1-x_{ik}) p_{jk}$$

$$\therefore p_{jk} = \frac{\sum_{i=1}^{m} \lambda_{ij} x_{ik}}{\sum_{i=1}^{m} \lambda_{ij}}$$

To find θ_j we need to maximize $h(\theta,p)$ or equivalently maximize $\sum_{i=1}^{m} \sum_{j=1}^{K} \lambda_{ij} log\theta_j$ with the constraint $\sum_{j=1}^{K} \theta_j = 1$, for which we use the lagrange multiplier method.

$$\hat{l} = \sum_{i=1}^{m} \sum_{j=1}^{K} \lambda_{ij} log\theta_j - \beta(\sum_{j=1}^{K} \theta_j - 1) \implies \frac{\delta \hat{l}}{\delta \theta_j} = (\sum_{i=1}^{n} \frac{\lambda_{ij}}{\theta_j}) - \beta = 0 \implies \theta_j = \frac{\sum_{i=1}^{m} \lambda_{ij}}{\beta}$$

$$\sum_{j=1}^{K} \theta_j = 1 \implies \sum_{j=1}^{K} \sum_{i=1}^{m} \frac{\lambda_{ij}}{\beta} = 1 \implies \beta = \sum_{i=1}^{m} \sum_{j=1}^{K} \lambda_{ij} = \sum_{i=1}^{m} 1 = m$$

$$\therefore \theta_j = \frac{1}{m} \sum_{i=1}^{m} \lambda_{ij}$$

Now fix p_{jk} and θ_j and maximize

$$\sum_{i=1}^{m}\sum_{j=1}^{K}\lambda_{ij}log\theta_j + \sum_{i=1}^{m}\sum_{j=1}^{K}\lambda_{ij}\gamma_{ij} - \sum_{i=1}^{m}\sum_{j=1}^{K}\lambda_{ij}log\lambda_{ij}$$

over λ_{ij} with constraint $\sum_{j=1}^{K}\lambda_{ij} = 1$ and where

$$\gamma_{ij} = \sum_{k=1}^{n} x_{ik}logp_{jk} + (1-x_{ik})log(1-p_{jk}) = log(\prod_{k=1}^{n} p_{jk}^{x_{ik}}(1-p_{jk})^{(1-x_{ik})})$$

Using lagrange multiplier method

$$\hat{l} = \sum_{i=1}^{m}\sum_{j=1}^{K}\lambda_{ij}log\theta_j + \sum_{i=1}^{m}\sum_{j=1}^{K}\lambda_{ij}\gamma_{ij} - \sum_{i=1}^{m}\sum_{j=1}^{K}\lambda_{ij}log\lambda_{ij} - \sum_{i=1}^{m}\beta_i(\sum_{j=1}^{K}\lambda_{ij} - 1)$$

$$\frac{\delta\hat{l}}{\delta\lambda_{ij}} = log\theta_j + \gamma_{ij} - log\lambda_{ij} - 1 - \beta_i = 0 \implies log\frac{\theta_j}{\lambda_{ij}} = \beta_i + 1 - \gamma_{ij} \implies \lambda_{ij} = \theta_j e^{\gamma_{ij}} e^{-\beta_i - 1}$$

$$\sum_{j=1}^{K}\lambda_{ij} = 1 \implies \sum_{j=1}^{K}\theta_j e^{\gamma_{ij}} e^{-\beta_i - 1} = 1 \implies e^{\beta_i + 1} = \sum_{j=1}^{K}\theta_j e^{\gamma_{ij}}$$

$$\therefore \lambda_{ij} = \frac{\theta_j e^{\gamma_{ij}}}{\sum_{l=1}^{K}\theta_l e^{\gamma_{il}}}$$

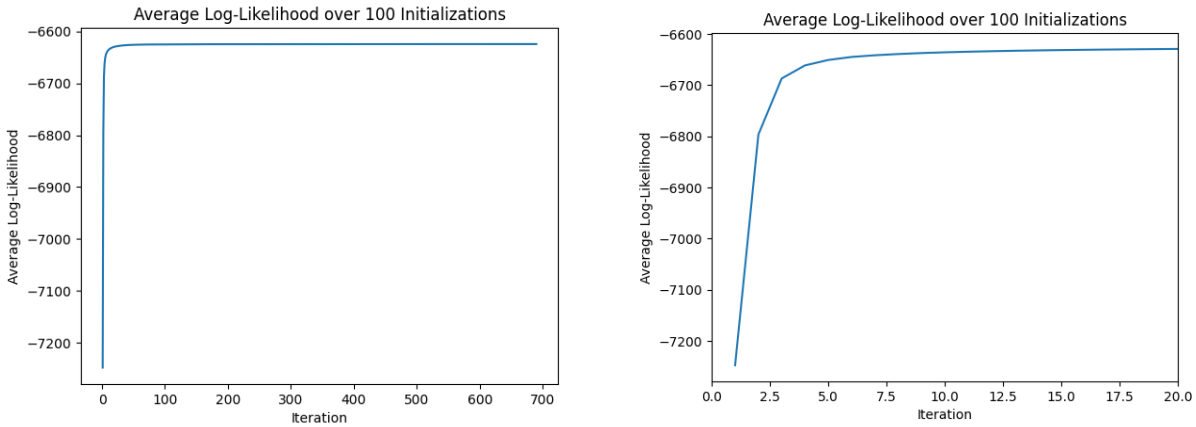
where

$$\gamma_{ij} = log(\prod_{k=1}^{n} p_{jk}^{x_{ik}}(1-p_{jk})^{(1-x_{ik})}) \implies e^{\gamma_{ij}} = \prod_{k=1}^{n} p_{jk}^{x_{ik}}(1-p_{jk})^{(1-x_{ik})}$$

This gives us the algorithm to compute the parameters of the Bernoulli mixture model.
The EM Algorithm :

- Initialize $\theta_1, ..., \theta_K, p_{(1,1)}, ..., p_{(K,n)}$ from the distribution $U(0,1]$. To satisfy the constraint $\sum_{j=1}^{K}\theta_j = 1$ normalize the values of θ_j.
- Do until convergence condition: $||\theta^{t+1} - \theta^t|| \leq \epsilon \; and \; ||p^{t+1} - p|| \leq \epsilon$
  - E-step: Fix θ_j and p_{jk} and compute λ_{ij}
  - M-step: Fix λ_{ij} and compute θ_j and p_{jk}

The file '1a.py' implements this algorithm and runs it for $K = 4$ and for 100 random initializations. (Note: The code will run until the algorithm converges, i have not included a condition for the algorithm to stop after set number of iterations. For the given dataset the code ran for upto an hour.)



(a) Plotted for upto 700 iterations

(b) Plotted for upto 20 iterations

Figure 2: Averaged Log-Likelihood over 100 random initializations

Both the figures are plotted using the same averaged log likelihood data but Figure 2(b) was plotted only for upto 20 iterations for a better visualization of the trend. We can see that on an average after 10 iterations the likelihood has not increased significantly.

(ii)

Run the K-means algorithm with $K = 4$ on the same data. Plot the objective of K-means as a function of iterations.

Ans: The file '1b.py' contains the code that implements the Lloyd's algorithm on the given dataset $X_{50\times m}$ for $K = 4$.
The Lloyd's Algorithm :

- Assign each data point uniformly at random to one of the k clusters.
- Do until convergence: Compute the mean of each cluster and assign it as the cluster's center, assign each data point to a cluster whose distance to the center is minimum.
- The algorithm converges when the cluster assignment in the previous iteration is same as the cluster assignment in the current iteration

The code computes the objective function

$$F(z_1, ..., z_m) = \sum_{i=1}^{m} ||x_i - \mu_{z_i}||_2^2$$

and plots it.

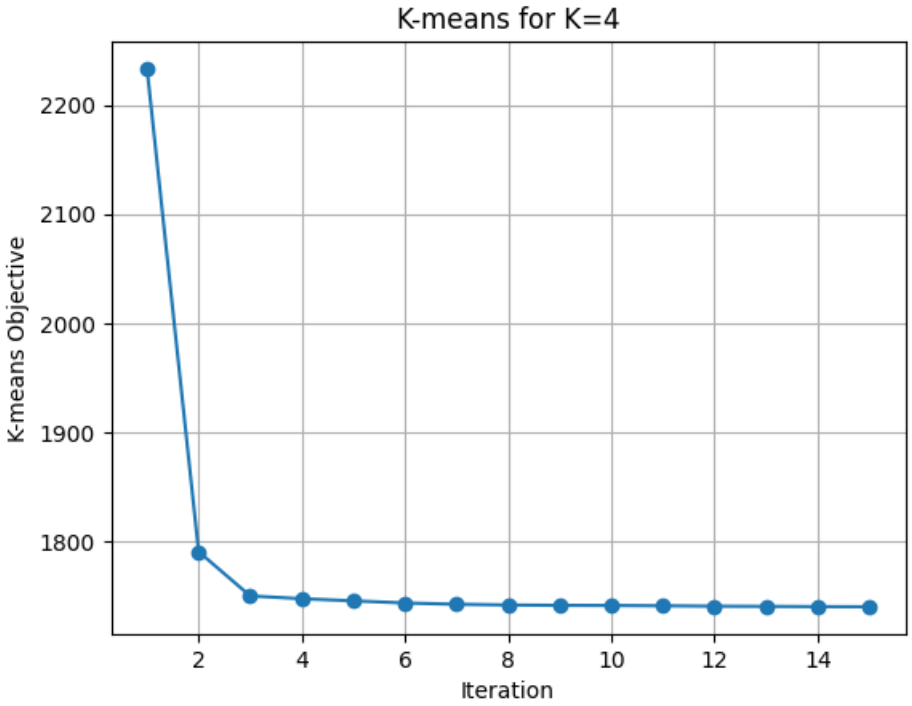


Figure 3: K-means objective vs Iterations

We can see that after 4 iterations the K-means objective has not decreased significantly.

(iii)

Among the two different algorithms implemented above, which do you think you would choose for this dataset and why?

Ans: EM algorithm gives us soft clusters whereas K-means gives hard clusters. We can still obtain hard clusters from EM and soft clusters K-means. To get hard clusters from EM just assign the point to the cluster with the highest probability of the point being generated from. To get soft clusters from K-means use the clusters formed as samples for the mixtures and get the required sample statistics as parameters for the density function. Hence whether our goal is to obtain hard clusters or soft clusters we can use either of the two algorithms.
But as we saw in (i) ans (ii) the K-means algorithm takes less time and less resources as compared to EM. Thus if we are limited in time and resources K-means is a better choice than EM.
For this problem we want to maximize the likelihood of the observed data, hence i compare the average log likelihood of both EM and K-means for 5 random initializations and $K = 4$. The file '1c_p1.py' contains code that runs EM for 5 random initializations and computes the average log likelihood, which turned out to be -6624.7698.

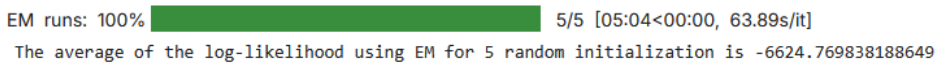


Figure 4: Output of 1c_1p.py

The file '1c_p2.py' contains code that runs K-means for 5 random initializations. After the clusters are formed it computes θ_j and p_{jk} as $\theta_j = \frac{No.\ of\ points\ in\ cluster\ j}{m}$ and $p_{jk} = \frac{Sum\ of\ all\ k^{th}\ entries\ of\ points\ in\ cluster\ j}{No.\ of\ points\ in\ cluster\ j}$. See that $p_j = \begin{bmatrix} p_{(j,1)} & \dots & p_{(j,n)} \end{bmatrix} = \mu_j = center\ of\ cluster\ j$. Using these parameters it computes the average log likelihood, which turned out to be -7654.1042.

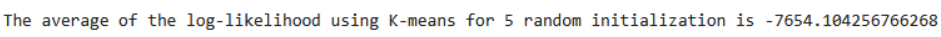


Figure 5: Output of 1c_2p.py

Thus for this dataset if our priority is to maximize the likelihood over computational cost then I would choose the EM algorithm since on average it gives larger likelihood than K-means algorithm.

1 Question 2

You are given a data-set in the file `A2Q2Data_train.csv` with 10000 points in $\mathbb{R}^{100} \times \mathbb{R}$. Each row corresponds to a data point where the first 100 components are features and the last component is the associated y value.

(i)

Obtain the least squares solution w_{ML} to the regression problem using the analytical solution.

Ans: Given $X_{n\times m}$ and $y_{m\times 1}$, each column of X is a data point with its associated label in y. If XX^T is invertible then the least squares solution w_{ML} is given by

$$w_{ML} = (XX^T)^{-1}Xy$$

The file '2a.py' contains the code which computes this expression. It also contains code to compute the error on test set due to w_{ML} which is required for a later question. It also saves the vector w_{ML} as an 'npy' file, which is also required for solving later questions.

```
The estimate for w using ordianry least squares is:

[-7.84961008e-03 -1.36715320e-02 -3.61656438e-03  2.64909162e-03
  1.88551446e-01  2.65314657e-03  9.46531787e-03  1.79809482e-01
  3.73757317e-03  4.99608944e-01  8.35836264e-03  4.29108775e-03
  1.42141179e-02  3.94232414e-03  9.36795890e-03 -1.12038274e-03
  3.35727498e-03  1.16152213e-03 -9.40884704e-03 -2.45575474e-03
 -1.17409629e-02 -1.01960612e-02  7.95771320e-03 -1.00574853e-02
  6.04882939e-03 -4.67345193e-03 -3.09091547e-03  8.14909191e-03
  1.20264599e-02 -6.82458167e-03 -8.65405539e-03  9.86273489e-04
  4.92968012e-03  5.99772460e-03 -1.34667859e-02  1.07075730e-03
  1.32745992e-02 -1.14148742e-02 -2.01056697e-02  5.85096240e-01
  4.94483255e-04 -7.86666916e-04 -2.71926574e-03 -9.54021938e-03
 -5.44161059e-03  9.80679209e-03 -6.72540624e-03 -4.45414300e-04
  6.98516510e-03  3.16138907e-02  4.51763485e-01 -8.75221378e-03
  2.55167390e-03  4.24921149e-03  2.89847927e-01  7.03723254e-03
 -1.95796945e-03  1.41523883e-02 -1.06508170e-02  7.72743903e-01
 -5.67126047e-03 -6.30026187e-04  6.50943015e-03 -4.84019165e-03
  4.63832328e-03  4.54887178e-03 -2.99475114e-03  8.38781696e-03
 -2.47558718e-03  9.00947917e-04  1.14713511e-03 -1.87641341e-03
 -1.05175760e-02 -9.31304110e-03 -1.23550002e-03  5.97797559e-01
 -4.78625012e-03 -1.13727852e-02  2.88477059e-03  8.48999776e-01
 -1.08924235e-02  2.26346489e-03 -1.38099798e-03 -6.35934691e-03
  5.83784109e-03  5.69286755e-03  5.35566859e-03 -8.20616315e-03
  1.29884015e-02 -2.30575633e-03 -1.22263772e-04  8.66629173e-03
 -4.29446298e-03  5.69510898e-03  7.55483352e-03 -9.43540843e-03
  1.82905446e-02 -1.16998885e-03 -2.61599135e-03 -8.58616116e-03]
```

Figure 6: The least squares solution w_{ML}

(ii)

Code the gradient descent algorithm with a suitable step size to solve the least squares problem and plot $\|w_t - w_{\mathrm{ML}}\|_2$ as a function of t. What do you observe?

Ans: The goal is to compute w that minimizes the error

$$E(w) = \sum_{i=1}^{m} (w^T x_i - y_i)^2 = (X^T w - y)^T (X^T w - y)$$

We will use the Gradient descent algorithm to find w that minimizes $E(w)$.
The Gradient Descent Algorithm :

- Randomly initialize the vector w
- Do until convergence condition: $\nabla E(w) < \epsilon$
  - $w_{t+1} = w_t - \eta \nabla E(w)$

$\nabla E(w)$ is the gradient of the function $E(w)$ at w. η is the step size. η can be static or dynamic, but in my implementation i chose a standard static η as $\eta = \frac{1}{L}$, where L is $2 \times \|XX^T\|_2^2$. It turned out $\eta = 2 \times 10^{-6}$. The gradient is computed by

$$\nabla E(w) = 2(XX^T w - Xy)$$

The file '2b.py' contains code that implements this algorithm to compute w. The w obtained after convergence condition is the estimate for the least squares solution w_{GD}.

```
The least squares estimator obtained by gradient descent is :w_GD=
[-7.84791893e-03 -1.36631798e-02 -3.62928334e-03  2.64071456e-03
  1.88543532e-01  2.64726469e-03  9.46453353e-03  1.79804216e-01
  3.74808085e-03  4.99605843e-01  8.35749695e-03  4.29162110e-03
  1.42162618e-02  3.93505724e-03  9.36148271e-03 -1.12253462e-03
  3.35519624e-03  1.17134766e-03 -9.40780380e-03 -2.44562060e-03
 -1.17363173e-02 -1.01832867e-02  7.96264874e-03 -1.00581438e-02
  6.04957201e-03 -4.67304166e-03 -3.09843205e-03  8.14936839e-03
  1.20210971e-02 -6.81935555e-03 -8.65819952e-03  9.88525352e-04
  4.93270049e-03  5.99796921e-03 -1.34689881e-02  1.07006698e-03
  1.32758552e-02 -1.14189225e-02 -2.00928321e-02  5.85087470e-01
  4.92036357e-04 -7.82357171e-04 -2.72677262e-03 -9.53479047e-03
 -5.43634962e-03  9.80765034e-03 -6.73427108e-03 -4.44676513e-04
  6.98237303e-03  3.16206574e-02  4.51757344e-01 -8.75308575e-03
  2.55918329e-03  4.25079480e-03  2.89845547e-01  7.03683865e-03
 -1.94968924e-03  1.41522747e-02 -1.06506032e-02  7.72728188e-01
 -5.66659081e-03 -6.24895004e-04  6.51858956e-03 -4.83191116e-03
  4.64513509e-03  4.55092322e-03 -2.99810002e-03  8.39497775e-03
 -2.47877447e-03  8.91025886e-04  1.15476415e-03 -1.88458857e-03
 -1.05170844e-02 -9.31755756e-03 -1.21982163e-03  5.97788954e-01
 -4.77694059e-03 -1.13626484e-02  2.89427858e-03  8.48982318e-01
 -1.08909377e-02  2.27391081e-03 -1.37870704e-03 -6.35344169e-03
  5.83323169e-03  5.68951595e-03  5.35476442e-03 -8.21163359e-03
  1.29954486e-02 -2.31079352e-03 -1.32674205e-04  8.66911456e-03
 -4.30330822e-03  5.67915642e-03  7.54557211e-03 -9.42773189e-03
  1.82819275e-02 -1.16322163e-03 -2.61283138e-03 -8.59181362e-03]
```

Figure 7: The least squares solution obtained using Gradient descent

The code also computes how close w gets to w_{ML} after every iteration with respect to the $l2$ norm.

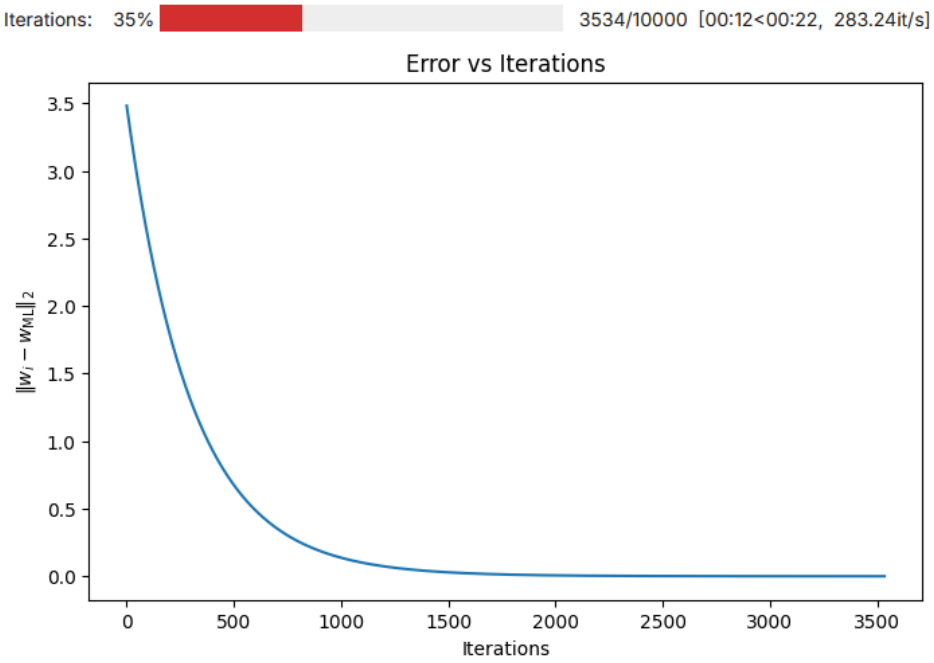


Figure 8: Error between w and w_{ML} per iteration

We can see that the error decreases monotonically and after around 1000 iterations there is no significant decrease of the error between the two. The algorithm terminated after 3534 iterations.

```
The error between w_ML and w_GD is: 6.846430639533136e-05
```

Figure 9: Error between w_{GD} and w_{ML}

We can observe that the error between our estimate w_{GD} and the actual w_{ML} is around 6.8464×10^{-5} which is very small. Hence w_{GD} converges to a point very close to w_{ML}.

(iii)

Code the stochastic gradient descent algorithm using a batch size of 100 and plot $\|w_t - w_{\text{ML}}\|_2$ as a function of t. What are your observations?

Ans: Stochastic gradient method is a variation of the gradient descent method in which we randomly sample a small batch of data from the dataset and treat it as the entire data set for that iteration.
The Stochastic Gradient Descent Algorithm :

- Randomly Initialize the vector w
- Do until convergence condition: $\nabla E(w) < \epsilon$
  - Randomly sample k columns of $X_{n\times m}$ to create $\hat{X}_{n\times k}$. Create $\hat{y}_{k\times 1}$ by choosing the labels associated with the k columns.
  - $w_{t+1} = w_t - \eta \nabla E(w)$. Where $\nabla E(w) = 2(\hat{X}\hat{X}^T w_t - \hat{X}\hat{y})$

In general people use stochastic gradient descent when m is very large, i.e. computing XX^T and Xy is expensive. Hence the previous choice of $\eta = \frac{1}{L}$ which involved computing XX^T cannot be used. After some runs with different choices of η, i chose 2×10^{-4} for this problem.
The w obtained after the iteration ends can be a choice for the least squares estimator, but generally people choose $w_{SGD} = \frac{1}{T}\sum_{t=1}^{T} w_t$ as the least squares estimator.
The file '2c.py' contains the code that implements this algorithm and computes both w at the end of iteration and w_{SGD}.

```
The weight vector after the last update in the SGD algorithm is :w=
[-9.10960073e-03 -1.11332197e-02 -5.59932538e-04  3.06087764e-03
  1.83670928e-01  6.11952854e-03  1.03673636e-02  1.81385414e-01
  8.11646519e-03  4.96287517e-01  6.75743179e-03  8.11856388e-03
  1.63449738e-02  2.05118004e-03  7.95641112e-03 -2.64646215e-03
 -5.24782124e-04 -1.74763118e-03 -1.04385685e-02 -3.38599429e-03
 -9.52736580e-03 -1.35185480e-02  1.15299062e-02 -9.81100461e-03
  5.39212999e-03 -4.90956663e-03 -4.89901696e-03  9.67327982e-03
  1.00704755e-02 -4.68119840e-03 -9.54678116e-03 -3.03056335e-03
  5.17129747e-03  7.68726138e-03 -7.04797023e-03 -6.70075420e-03
  1.44129324e-02 -8.91428723e-03 -2.15607531e-02  5.82912101e-01
  6.65858385e-04 -2.91944943e-03 -2.44740781e-03 -5.30738785e-03
 -8.86910973e-06  4.67501408e-03 -6.06419605e-03 -4.16228733e-04
  8.03912139e-03  3.02346680e-02  4.50204254e-01 -5.88528212e-03
  4.59413336e-03  2.59864810e-03  2.86113628e-01  8.70318011e-03
 -1.24484035e-03  1.35137302e-02 -9.07847819e-03  7.69951895e-01
 -1.25510326e-03 -3.69216262e-03  1.20660314e-02 -6.06931719e-03
  6.55588806e-03  4.51913418e-03 -3.23324211e-03  1.08613906e-02
 -7.40575263e-03 -3.91046852e-03  7.91917961e-03 -2.36272000e-03
 -1.12087611e-02 -6.44565698e-03 -1.88011320e-03  5.98002917e-01
 -4.00164214e-03 -1.09207656e-02  2.56666365e-03  8.49659167e-01
 -1.02311433e-02 -8.02226136e-04  7.16778393e-04 -5.10205206e-03
  6.18882720e-03  3.63822087e-03  8.26907187e-03 -1.25098782e-02
  1.56150509e-02 -1.71295386e-03 -1.60005807e-03  4.43721677e-03
 -1.03513765e-03  5.21091405e-03  1.15976195e-02 -6.34618846e-03
  2.26341964e-02 -1.27776759e-04 -2.97118314e-03 -1.41275570e-02]
```

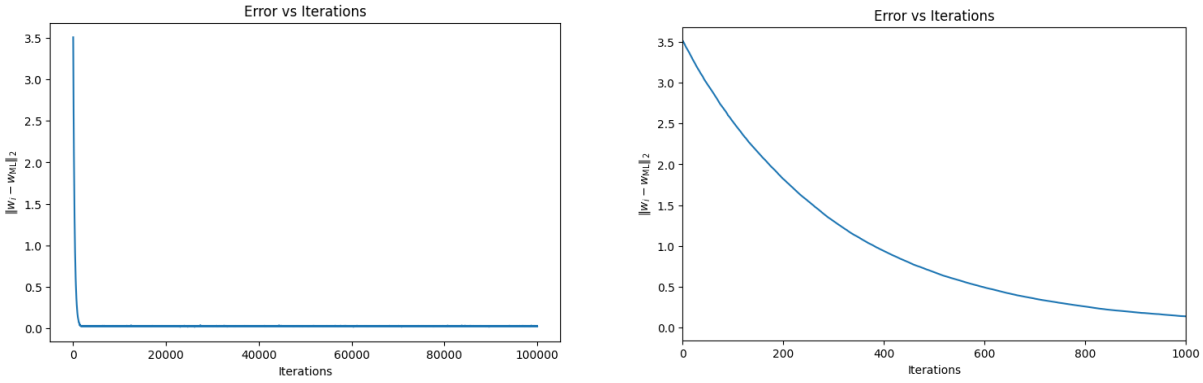
(a) w at the end of iteration

```
The least squares estimator obtained by stochastic gradient descent is :w_SGD=
[-8.71290057e-03 -1.27143259e-02 -4.20751234e-03  1.42374628e-03
  1.87273868e-01  1.64446642e-03  1.01304625e-02  1.80383132e-01
  5.46947589e-03  4.99177778e-01  7.49175542e-03  4.04074692e-03
  1.42674304e-02  3.47235629e-03  8.09543837e-03 -9.20588546e-04
  2.40381102e-03  1.39676833e-03 -9.52955018e-03 -1.29446888e-03
 -1.21509967e-02 -8.63904879e-03  8.81033328e-03 -9.56428766e-03
  6.73083259e-03 -4.01847976e-03 -3.87496825e-03  8.85873676e-03
  1.17884740e-02 -5.95895320e-03 -9.64413355e-03  1.25871498e-03
  5.73145340e-03  6.44848021e-03 -1.37175943e-02  1.10568004e-04
  1.35887498e-02 -1.22464706e-02 -1.93590419e-02  5.83712079e-01
 -7.84009226e-04 -1.51508656e-03 -4.03870073e-03 -7.85579497e-03
 -4.93683690e-03  8.82583469e-03 -6.83663360e-03  9.27689427e-04
  6.79272873e-03  3.28984724e-02  4.49739884e-01 -8.24165434e-03
  4.29227038e-03  5.31006506e-03  2.90013565e-01  7.63662229e-03
 -3.58652362e-04  1.46354777e-02 -1.04546676e-02  7.69421178e-01
 -4.64761746e-03 -1.89356181e-03  7.27015559e-03 -2.89531975e-03
  6.02208432e-03  5.68734272e-03 -2.67657022e-03  8.38982186e-03
 -3.60838709e-03  9.75030078e-04  2.20415088e-03 -2.24500441e-03
 -1.14243401e-02 -1.00899667e-02  8.64848739e-04  5.94975930e-01
 -3.91173910e-03 -1.18607184e-02  4.25453547e-03  8.46020878e-01
 -1.05430871e-02  2.53596749e-03 -1.47669986e-03 -5.09879405e-03
  4.45014003e-03  5.82872728e-03  4.95034140e-03 -8.53906232e-03
  1.38257606e-02 -2.59655441e-03  4.11045141e-04  1.04499460e-02
 -4.50848732e-03  4.34721119e-03  6.74260829e-03 -9.77107003e-03
  1.74155912e-02 -1.54100692e-03 -1.71325904e-03 -9.50392788e-03]
```

(b) w_{SGD}

Figure 10: The Least squares estimator using SGD algorithm

The code also computes how close w gets to w_{ML} after every iteration.



(a) Plotted for 100000 iterations (b) Plotted for 1000 iterations

Figure 11: Error between w and w_{ML} for after every iteration

Both the plots show the same data except that Figure 11(b) plots it only for 1000 iterations for a better visualization of the trend.
We can see that the error monotonically decreases for every iteration but after a 1000 iterations there not a significant decrease.

```
The error between w_ML and w is: 0.02802963705390463
The error between w_ML and w_sgd is: 0.010670429177212996
```

Figure 12: Error between w after last update vs w_{ML} and w_{SGD} vs w_{ML}

We can observe that even after the algorithm ran for 10^5 iterations the least squares estimators obtained are not nearly as close to w_{ML} as w_{ML} was close to w_{GD} after 3534 iterations, which is expected since the SGD computes gradient on a smaller dataset than the entire one. But see that in both the GD and SGD algorithms w didn't change significantly after 1000 iterations, hence we could have stopped both the algorithms and ended up with errors (between w and w_{ML}) of approximately 6.8464×10^{-6} for GD and 0.02803 SGD. And for practical purposes this error of 0.02803 is perfectly reasonable tradeoff for less computational resources.
We can also observe that in this case w_{SGD} is better estimator of w_{ML} than w after the last update.

(iv)

Code the gradient descent algorithm for ridge regression. Cross-validate for various choices of λ and plot the error in the validation set as a function of λ.

For the best λ chosen, obtain w_R. Compare the test error (for the test data in the file `A2Q2Data_test.csv`) of w_R with w_{ML}. Which is better and why?

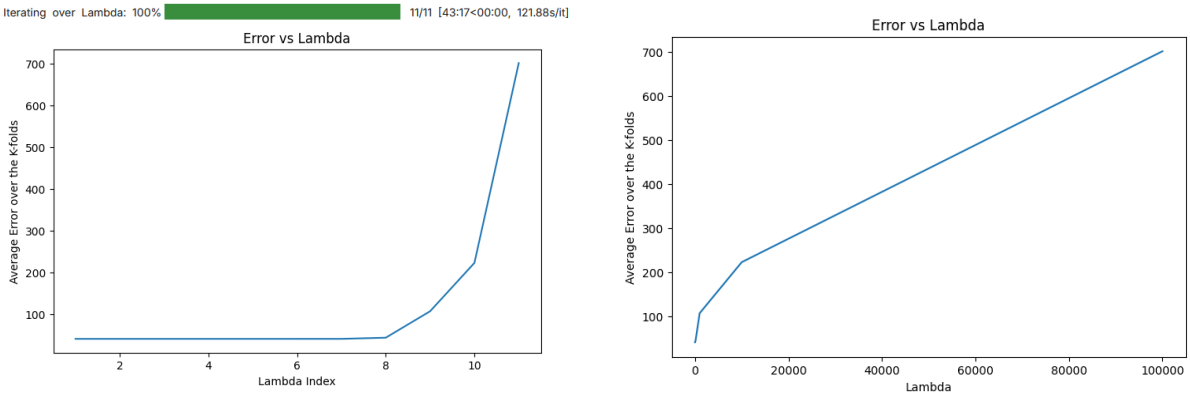
Ans: Unlike linear regression that aims to minimize $E(w) = \|XX^Tw - Xy\|_2^2$, ridge regression aims to minimize the function,

$$E_R(w) = \|(XX^T + \lambda I)w - Xy\|_2^2$$

for a fixed λ. Ridge regression tries to strike a balance between minimizing $E(w)$ and pushing w towards 0. There exists some λ such that $E_R(w_R) < E(w_{ML})$, but finding that λ is difficult. In this problem I search for the best λ in the set $\Lambda = \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2, 10^3, 10^4, 10^5\}$ using the cross validation method.
The Cross Validation Method :

- Create K folds, that is create K partitions of the Dataset
- For each λ in Λ
  - For $i = 1$ to K
    - Keep the ith fold for validation. Concatenate the remaining $K - 1$ folds into X_T and y_T.
    - Compute w_i using gradient descent on the dataset X_T and y_T for the function $E_R(w)$. $\nabla E_R(w) = 2(XX^Tw - Xy + \lambda w)$
    - Compute the error $E(w_i)$ on the ith fold using w_i
  - Compute the average error $E_\lambda = \frac{1}{K}\sum_{i=1}^{K} E(w_i)$
- Choose λ with the least E_λ

The file '2d_p1.py' contains the code which finds the best choice of λ from Λ using the cross validation method. In this implementation I chose $K = 10$. For η in the gradient descent algorithm, I chose to make it dynamic with respect to λ. Because if i keep it static, for larger values of λ the gradient will take bigger steps for the same number of iterations. I chose $\eta_\lambda = \frac{1}{L+\lambda}$ where $L = 5 \times 10^5$.



(a) Error against index of λ in Λ (b) Error against λ

Figure 13: Average error over the 10 folds for each λ in Λ

Both the plots show the same data except Figure 13(a) is plotted against index of λ in Λ for better visualization of trend since plotting it against λ makes the first half of Λ crowded together.

```
Lambda for which the average error is minimum is: 1
```

Figure 14: Best λ in Λ

The best choice for λ in Λ is found to be 1.
If $(XX^T + I)^{-1}$ is invertible then the ridge estimator w_R for $\lambda = 1$ is computed using

$$w_R = (XX^T + I)^{-1}Xy$$

The file '2d_p2.py' contains code that computes this expression.

```
The ridge regression estimate for w:

[-7.78349534e-03 -1.35983864e-02 -3.58360634e-03  2.68479640e-03
  1.88348208e-01  2.70219310e-03  9.49092079e-03  1.79624227e-01
  3.80837258e-03  4.99072893e-01  8.39458062e-03  4.33977508e-03
  1.42542751e-02  3.98423399e-03  9.41595899e-03 -1.09181265e-03
  3.42907347e-03  1.23962457e-03 -9.33825457e-03 -2.37000609e-03
 -1.16838140e-02 -1.01336982e-02  7.98643154e-03 -9.99846653e-03
  6.08831248e-03 -4.60502104e-03 -3.04820684e-03  8.18838451e-03
  1.20549078e-02 -6.75419946e-03 -8.59988450e-03  1.02068804e-03
  5.02063345e-03  6.02868964e-03 -1.33914960e-02  1.11088967e-03
  1.33244084e-02 -1.13222072e-02 -2.00357852e-02  5.84445087e-01
  5.38709390e-04 -7.26678094e-04 -2.65241566e-03 -9.48374862e-03
 -5.36840698e-03  9.82756366e-03 -6.69258171e-03 -4.06083767e-04
  7.01373869e-03  3.16220307e-02  4.51290930e-01 -8.69960529e-03
  2.60857892e-03  4.27332400e-03  2.89540371e-01  7.05642527e-03
 -1.89862500e-03  1.41721647e-02 -1.05663387e-02  7.71834481e-01
 -5.59851442e-03 -5.52248082e-04  6.58661775e-03 -4.76035908e-03
  4.67707334e-03  4.58192288e-03 -2.93443060e-03  8.44625781e-03
 -2.39470556e-03  9.17255330e-04  1.22291354e-03 -1.81529701e-03
 -1.04400134e-02 -9.24227568e-03 -1.13983449e-03  5.97142910e-01
 -4.70302870e-03 -1.12748926e-02  2.96226671e-03  8.47996613e-01
 -1.08254104e-02  2.33793379e-03 -1.31086051e-03 -6.34024531e-03
  5.88174760e-03  5.72284720e-03  5.38931920e-03 -8.14641146e-03
  1.30209713e-02 -2.25584138e-03 -8.74291689e-05  8.68660828e-03
 -4.26098115e-03  5.71903156e-03  7.59891582e-03 -9.33647303e-03
  1.83353098e-02 -1.09375209e-03 -2.56122667e-03 -8.50145806e-03]
```

Figure 15: The Ridge estimator w_R for $\lambda = 1$

The code also computes the squared error due to w_R on the test set, which is found to be around 185.152.

```
The error on test set due to the ridge regression estimator w is: 185.15198972820724
```

Figure 16: Squared error on test set due to w_R

The problem also asks us to compare the squared error on the test set due to w_{ML} and w_R. The file '2a.py' has a piece of code that computes the squared error on the test set due to w_{ML}, which turned out to be around 185.364.

```
The error on test set due to the MLE estimator w is: 185.36365558461083
```

Figure 17: Squared error on test set due to w_{ML}

We can see that the squared error on the test set due to w_R is slightly less than the error due to w_{ML}.
If we are only seeing the results on this test set we can say that w_R is slightly better than w_{ML} since w_R has given slightly less error on this unseen data. I would say both are equally good estimators since they give almost same error on the test set.